R3LIVE: A Robust, Real-time, RGB-colored, LiDAR-Inertial-Visual tightly-coupled state Estimation and mapping package

2022 (RA-L)

Jiarong Lin and Fu Zhang

2026.04.16

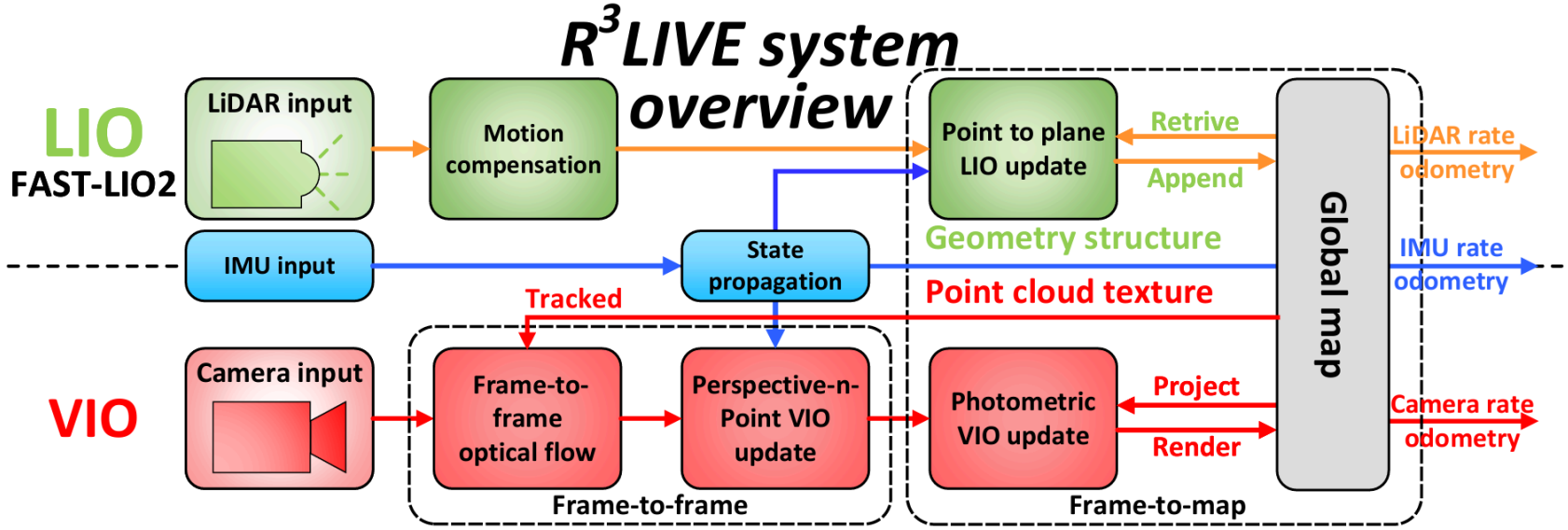
Contents

Introduction

- 과거로 돌아가보자...
- Gaussian Splatting으로 SLAM을 하기 이전에도, **Colorized point cloud**를 이용해 눈으로 보기에 충분히 사실적인 SLAM이 가능했음
- 저가형 solid-state LiDAR가 등장하면서, 기존 ring-channel LiDAR보다 훨씬 저렴하면서, dense함
- 다만, LiDAR만 사용하는 방법론들 자체가 feature라는걸 정의하기가 어려움.
  - 더욱이, **제한된 FoV**를 가진 solid-state LiDAR의 경우, feature가 부족한 상황이 자주 발생함
- 그래서 LiDAR와 Camera를 합친 방법이 두두등장!
  - 역설적이게도, 두 센서(solid-state LiDAR, Camera) 모두 제한된 FoV를 가지는 경우가 많음

Introduction

- 본 논문은, tightly coupled 방식!
- 크게 2가지의 contribution으로 나눌 수 있음
  - RGB-colored point cloud를 이용한 SLAM 방법론 제안
  - LIO, VIO를 subsystem으로 하여 tightly-coupled 방식으로 융합하는 방법



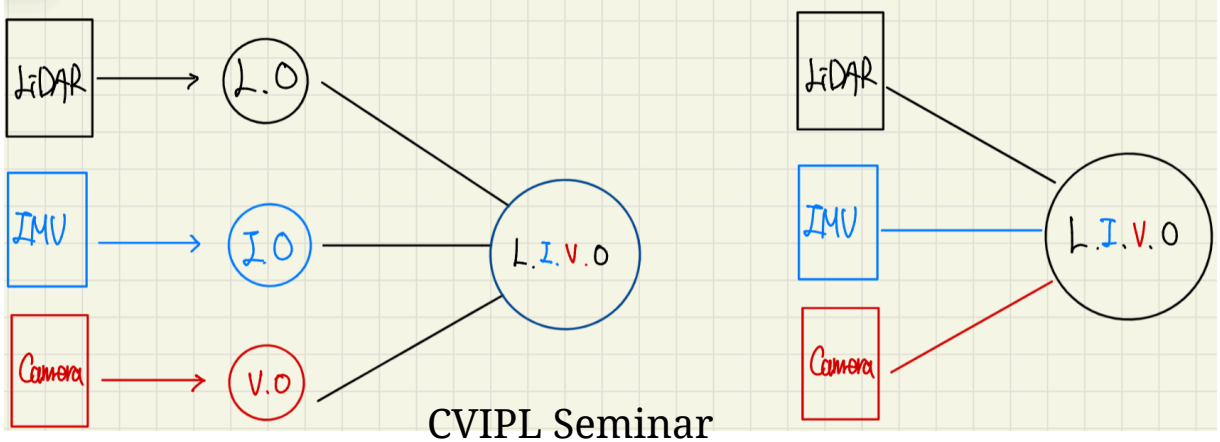
Background

- Loosly-coupled vs. tightly-coupled
- Point-to-plane residual
- Kalman Filter

Background

Loosly-coupled vs. tightly-coupled

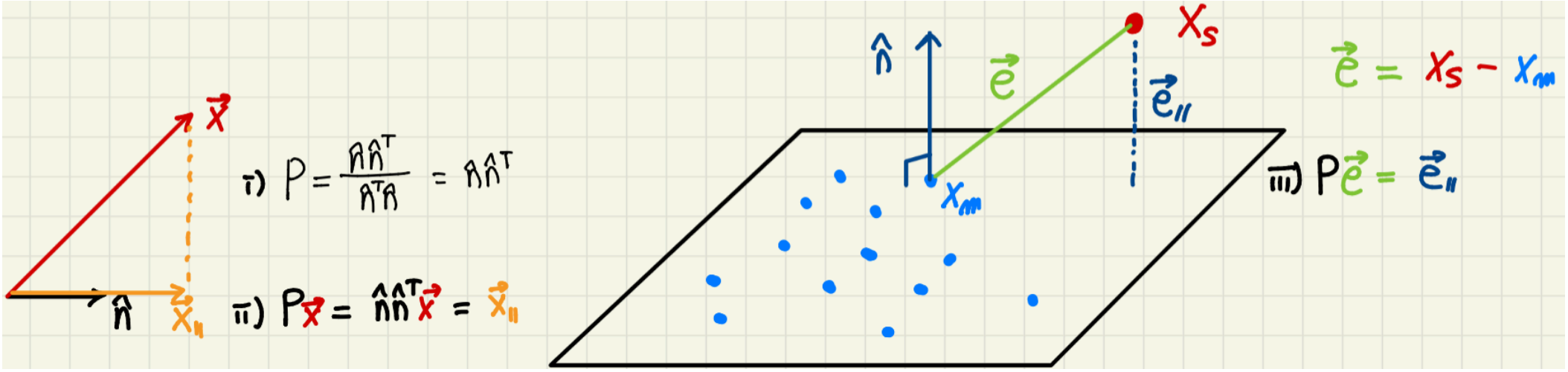
- Sensor fusion : loosly-coupled 방식과 tightly-coupled 방식으로 나뉨
- Loosly-coupled : 각 센서의 pose estimation을 독립적으로 수행한 후, 이를 융합
- Tightly-coupled : 각 센서의 raw measurement를 이용하여, 한번에 구함
- R3LIVE는 tightly-couple이지만, LIO, VIO가 각각 tightly couple(full-tightly는 아님!)



Background

Point-to-plane residual (Fast-LIO2)

- Point-to-plane residual : LiDAR의 point cloud에서, scan 된point가 기존의 map point로 구성한 plane으로부터 얼마나 떨어져 있는지



Background

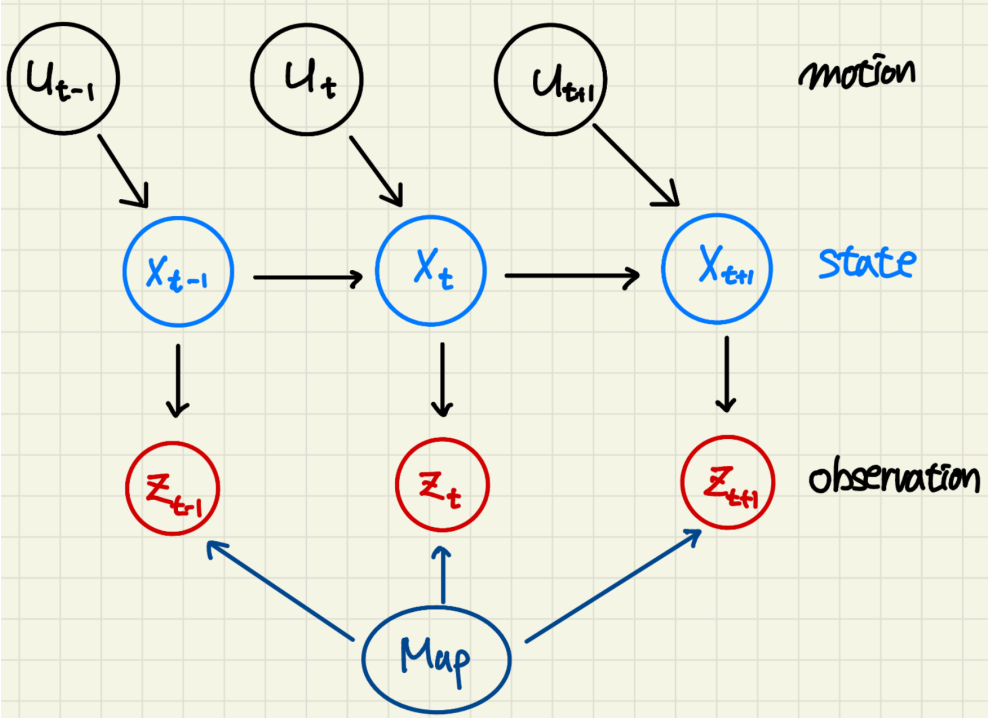
Kalman Filter - motion/observation model

- Kalman Filter는 motion model, observation model로 나눌 수 있음
- 기본적으로, Gaussian Distribution을 따른다고 가정
- Motion/Process/Prediction

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t) + \mathbf{w}_t$$

- Observation/Measurement/Correction

$$\mathbf{z}_{t,i} = h(\mathbf{x}_t, \mathbf{y}_i) + \mathbf{v}_{t,i}$$



Background

Kalman Filter - markov property

- 직전의 state에 대해서만 영향을 받음
- Motion model은 prediction, observation model은 correction step에 해당
- 즉, Kalman Filter는 prediction과 correction으로 이루어짐

Algorithm Kalman_filter$(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$**:**

$\bar{\mu}_t = A_t\ \mu_{t-1} + B_t\ u_t$

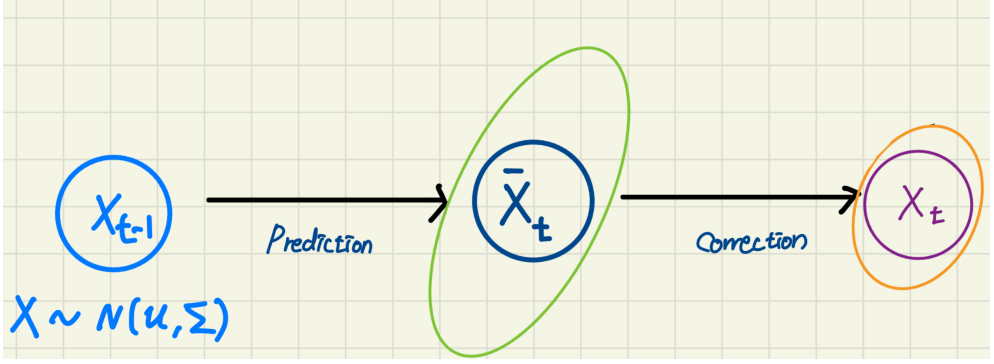
$\bar{\Sigma}_t = A_t\ \Sigma_{t-1}\ A_t^T + R_t$

$K_t = \bar{\Sigma}_t\ C_t^T (C_t\ \bar{\Sigma}_t\ C_t^T + Q_t)^{-1}$

$\mu_t = \bar{\mu}_t + K_t(z_t - C_t\ \bar{\mu}_t)$

$\Sigma_t = (I - K_t\ C_t)\ \bar{\Sigma}_t$

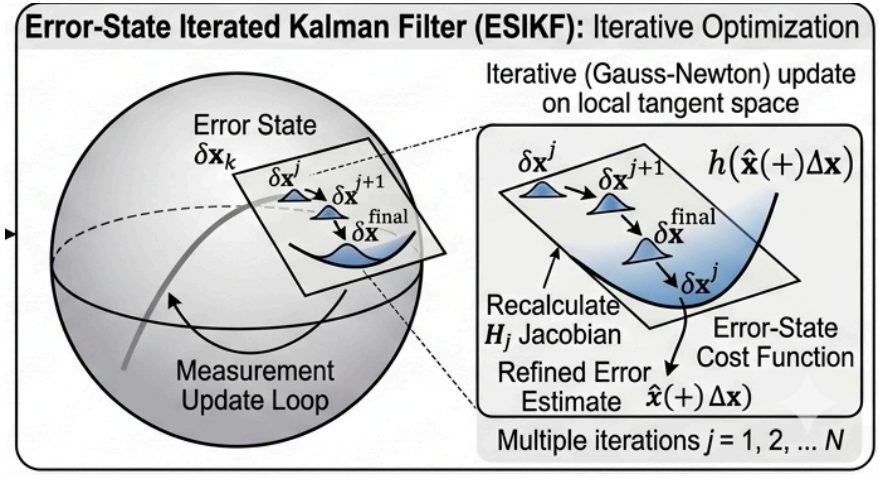
return μ_t, Σ_t



Background

ESIKF;Error State Kalman Filter

- Kalman Filter와 달리, ESIKF는 state를 nominal state와 local error state로 분리하여, error state를 iterative하게 추정하는 방식
  - 즉, State를 업데이트 할 때, prediction와 observation의 차이를 error로 두고, 이를 최소화 하는걸 iterative 하게!



Method

Overview

- R3LIVE는 LIO + VIO로 이루어져 있음
- Full state vector $\mathbf{x} \in \mathbb{R}^{29}$:

$$\mathbf{x} = \left[{}^{G}\mathbf{R}_{I}^{T},\ {}^{G}\mathbf{p}_{I}^{T},\ {}^{G}\mathbf{v}^{T},\ \mathbf{b}_{\mathbf{g}}^{T},\ \mathbf{b}_{\mathbf{a}}^{T},\ {}^{G}\mathbf{g}^{T},\ {}^{I}\mathbf{R}_{C}^{T},\ {}^{I}\mathbf{p}_{C}^{T},\ {}^{I}t_{C},\ \phi^{T} \right]^{T}$$

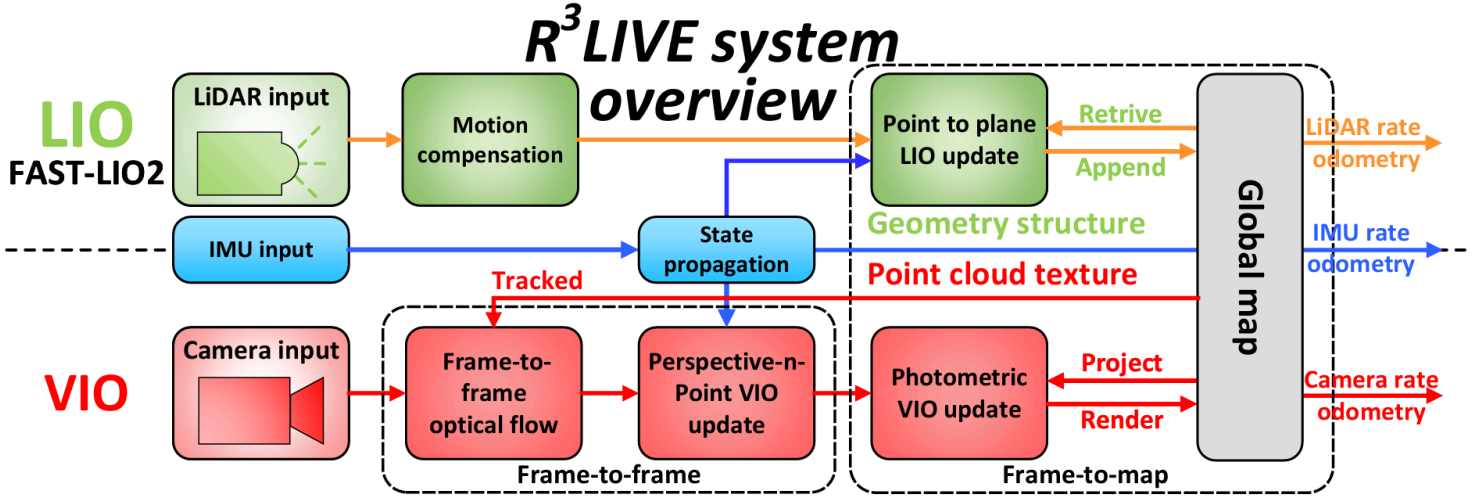
- R3LIVE는 10cm크기의 voxel에 해당 index 내의 point들을 저장
  - 과거의 point(map point)와 현재 scan 된 point를 각각 deactivated, activated로 구분
  - 각 point는 3D position과 RGB color로 표현(여기서, 각각(position, color)의 공분산도 들고 있음!)

$$\mathbf{P} = \left[{}^{G}\mathbf{p}_{x},\ {}^{G}\mathbf{p}_{y},\ {}^{G}\mathbf{p}_{z},\ \mathbf{c}_{r},\ \mathbf{c}_{g},\ \mathbf{c}_{b} \right]^{T} = \left[{}^{G}\mathbf{p}^{T},\ \mathbf{c}^{T} \right]^{T}$$

Method

LIO subsystem

- LIO subsystem은 Fast-LIO2를 기반으로 함
  - 그냥 Fast-LIO2임
- point cloud를 사용해 Geometry sturcture를 구성함(LIO-based SLAM)



Method

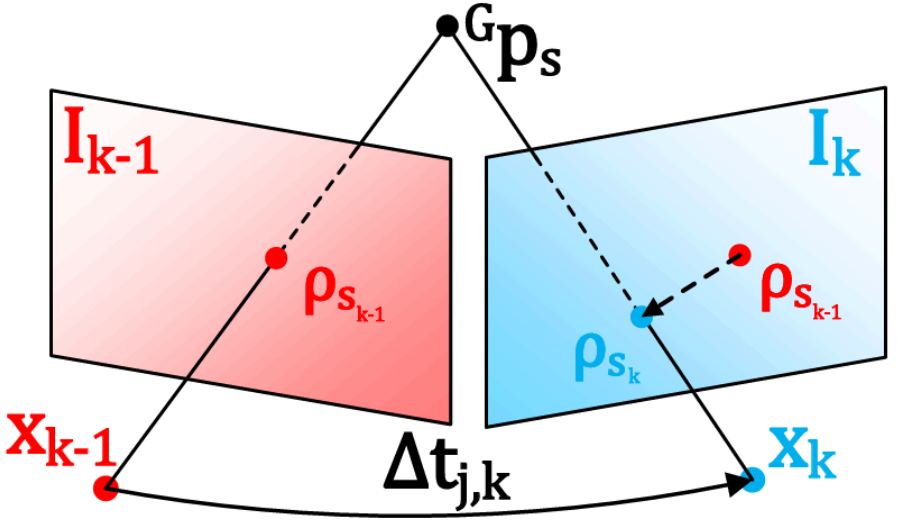
VIO subsystem

- 앞서 LIO를 이용하여 Geometry sturcture를 구성했음
  - 여기서, pose도 어느정도 정확하고, map point도 어느정도 정확함
  - sfm을 이용해 camra pose와 point를 얻었다고 생각하자!
- VIO에서는, 이 둘을 이용하여 point가 어떤 색을 가지는지, 이미지에서는 어떻게 보이는지 등을 이용해 residual을 구함
  - 즉, RGB color를 이용하여, residual을 구함
  - Gaussian Splatting, NeRF와 비슷해보이지만 다름,
  - 어떻게 하나 봅시다

Method

VIO subsystem - Frame-to-Frame residual

- Map에 있는 3D points :
  $\mathcal{P} = \{\mathbf{P}_1, \ldots, \mathbf{P}_m\}$
- 이전 이미지인 I_{t-1}번째 frame에 projection한 points :
  $\{\boldsymbol{\rho}_{1_{t-1}}, \ldots, \boldsymbol{\rho}_{m_{t-1}}\}$
- 현재 이미지 I_t번째 frame에 projection한 points :
  $\{\boldsymbol{\rho}_{1_t}, \ldots, \boldsymbol{\rho}_{m_t}\}$
- 이들의 위치는 Lucas-Kanade optical flow로 구함



Method

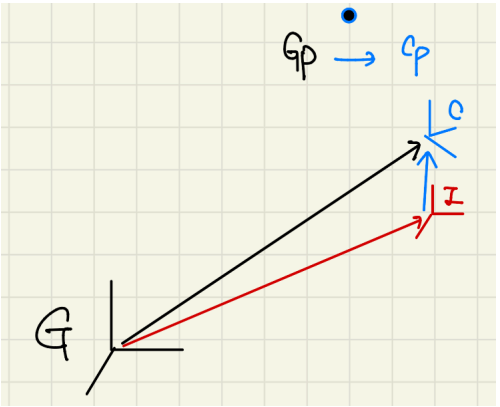
VIO subsystem - Frame-to-Frame residual

- Reprojection error :

$$\mathbf{r}\left(\check{\mathbf{x}}_k,\, \boldsymbol{\rho}_{s_k},\, {}^{G}\mathbf{p}_s\right) = \boldsymbol{\rho}_{s_k} - \boldsymbol{\pi}\left({}^{C}\mathbf{p}_s,\, \check{\mathbf{x}}_k\right)$$

$${}^{C}\mathbf{p}_s = \left[{}^{C}\mathbf{p}_{x_s},\, {}^{C}\mathbf{p}_{y_s},\, {}^{C}\mathbf{p}_{z_s}\right]^T = \left({}^{G}\check{\mathbf{R}}_{I_k} \cdot {}^{I}\check{\mathbf{R}}_{C_k}\right)^T \cdot {}^{G}\mathbf{p}_s - {}^{I}\check{\mathbf{R}}_{C_k}^T \cdot {}^{I}\check{\mathbf{p}}_{C_k} - \left({}^{G}\check{\mathbf{R}}_{I_k} \cdot {}^{I}\check{\mathbf{R}}_{C_k}\right)^T \cdot {}^{G}\check{\mathbf{p}}_{I_k}$$

$$\boldsymbol{\pi}\left({}^{C}\mathbf{p}_s,\, \check{\mathbf{x}}_k\right) = \left[\check{f}_{x_k}\frac{{}^{C}\mathbf{p}_{x_s}}{{}^{C}\mathbf{p}_{z_s}} + \check{c}_{x_k},\; \check{f}_{y_k}\frac{{}^{C}\mathbf{p}_{y_s}}{{}^{C}\mathbf{p}_{z_s}} + \check{c}_{y_k}\right]^T + \frac{{}^{I}\check{t}_{C_k}}{\Delta t_{k-1,k}}\left(\boldsymbol{\rho}_{s_k} - \boldsymbol{\rho}_{s_{k-1}}\right)$$



Method

VIO subsystem - Frame-to-Frame residual

- 정답처럼 보이는 map도, pixel에서의 point도 GT에 noise가 있다고 생각
  - 결국 추정된 값이기 때문에, GT에 noise가 더해진 형태

$$ {}^{G}\mathbf{p}_{s} = {}^{G}\mathbf{p}_{s}^{\mathrm{gt}} + \mathbf{n}_{\mathbf{p}_s}, \quad \mathbf{n}_{\mathbf{p}_s} \sim \mathcal{N}(\mathbf{0}, \mathbf{\Sigma}_{\mathbf{n}_{\mathbf{p}_s}}) $$

$$ \boldsymbol{\rho}_{s_k} = \boldsymbol{\rho}_{s_k}^{\mathrm{gt}} + \mathbf{n}_{\boldsymbol{\rho}_{s_k}}, \quad \mathbf{n}_{\boldsymbol{\rho}_{s_k}} \sim \mathcal{N}(\mathbf{0}, \mathbf{\Sigma}_{\mathbf{n}_{\boldsymbol{\rho}_{s_k}}}) $$

- 이상적인 조건이라면, position도, pixel도, 3차원의 점도 모두 residual이 0일것!
  - 하지만, 현재 추정된 pose, map point, pixel의 위치는 GT에 noise가 더해진 형태

$$ \mathbf{0} = \mathbf{r}(\mathbf{x}_k, \boldsymbol{\rho}_{s_k}^{\mathrm{gt}}, {}^{G}\mathbf{p}_{s}^{\mathrm{gt}}) \approx \mathbf{r}(\check{\mathbf{x}}_k, \boldsymbol{\rho}_{s_k}, {}^{G}\mathbf{p}_{s}) + \mathbf{H}_{s}^{r}\delta\check{\mathbf{x}}_k + \boldsymbol{\alpha}_s $$

Method

VIO subsystem - Frame-to-Frame residual

where $\boldsymbol{\alpha}_s \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_{\boldsymbol{\alpha}_s})$ and

$$\mathbf{H}_s^r = \left. \frac{\partial \mathbf{r}_c(\check{\mathbf{x}}_k \boxplus \delta\check{\mathbf{x}}_k,\ \boldsymbol{\rho}_{s_k},\ {}^G\mathbf{p}_s)}{\partial \delta\check{\mathbf{x}}_k} \right|_{\delta\check{\mathbf{x}}_k = \mathbf{0}}$$

$$\boldsymbol{\Sigma}_{\boldsymbol{\alpha}_s} = \boldsymbol{\Sigma}_{\mathbf{n}_{\rho_{s_k}}} + \mathbf{F}_{\mathbf{p}_s}^r \boldsymbol{\Sigma}_{\mathbf{p}_s} \mathbf{F}_{\mathbf{p}_s}^{r\,T}$$

$$\mathbf{F}_{\mathbf{p}_s}^r = \frac{\partial \mathbf{r}(\check{\mathbf{x}}_k,\ \boldsymbol{\rho}_{s_k},\ {}^G\mathbf{p}_s)}{\partial {}^G\mathbf{p}_s}$$

- 즉, error state로 편미분 한 것과, map point의 uncertianty, 그리고 KLT의 uncertainty를 더한 형태로 표현

Method

VIO subsystem - Frame-to-Frame VIO ESIKF update

- 앞서 nominal state일 때 residual을 구했음
- 이게 최적의 residual이 아니니, iterative하게 error state를 업데이트 하면서, residual을 줄이도록!

$$\min_{\delta\check{\mathbf{x}}_k}\left(\|\check{\mathbf{x}}_k \boxminus \hat{\mathbf{x}}_k + \mathcal{H}\delta\check{\mathbf{x}}_k\|^2_{\boldsymbol{\Sigma}_{\delta\hat{\mathbf{x}}_k}} + \sum_{s=1}^{m}\|\mathbf{r}\left(\check{\mathbf{x}}_k, \boldsymbol{\rho}_{s_k}, {}^G\mathbf{p}_s\right) + \mathbf{H}_s^r\delta\check{\mathbf{x}}_k\|^2_{\boldsymbol{\Sigma}_{\alpha_s}}\right) \tag{12}$$

Method

VIO subsystem - Frame-to-Frame VIO ESIKF update

$$\mathbf{H} = \left[\mathbf{H}_1^{r\,T}, \ldots, \mathbf{H}_m^{r\,T} \right]^T$$

$$\mathbf{R} = \operatorname{diag}(\boldsymbol{\Sigma}_{\boldsymbol{\alpha}_1}, \ldots, \boldsymbol{\Sigma}_{\boldsymbol{\alpha}_m})$$

$$\check{\mathbf{z}}_k = \left[\mathbf{r}\left(\check{\mathbf{x}}_k, \boldsymbol{\rho}_{1_k}, {}^G\mathbf{p}_1 \right) \cdots \mathbf{r}\left(\check{\mathbf{x}}_k, \boldsymbol{\rho}_{m_k}, {}^G\mathbf{p}_m \right) \right]^T$$

$$\mathbf{P} = (\mathcal{H})^{-1} \boldsymbol{\Sigma}_{\delta\hat{\mathbf{x}}_k} (\mathcal{H})^{-T}$$

Following [6], we have the Kalman gain computed as:

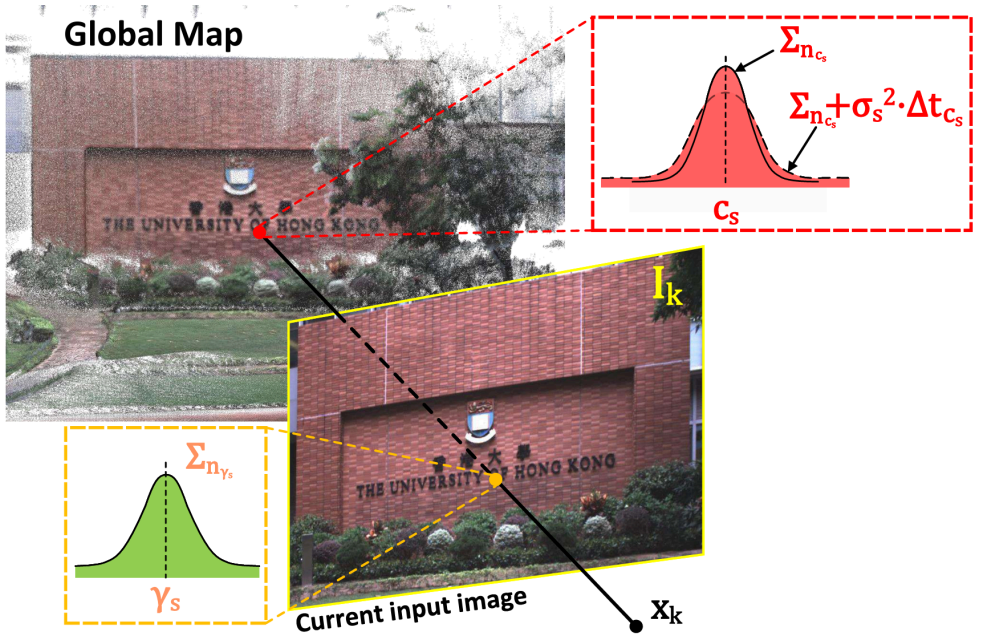
$$\mathbf{K} = \left(\mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} + \mathbf{P}^{-1} \right)^{-1} \mathbf{H}^T \mathbf{R}^{-1}$$

Then we can update the state estimate as:

$$\check{\mathbf{x}}_k = \check{\mathbf{x}}_k \boxplus \left(-\mathbf{K}\check{\mathbf{z}}_k - (\mathbf{I} - \mathbf{K}\mathbf{H})(\mathcal{H})^{-1} \left(\check{\mathbf{x}}_k \boxminus \hat{\mathbf{x}}_k \right) \right)$$

Method

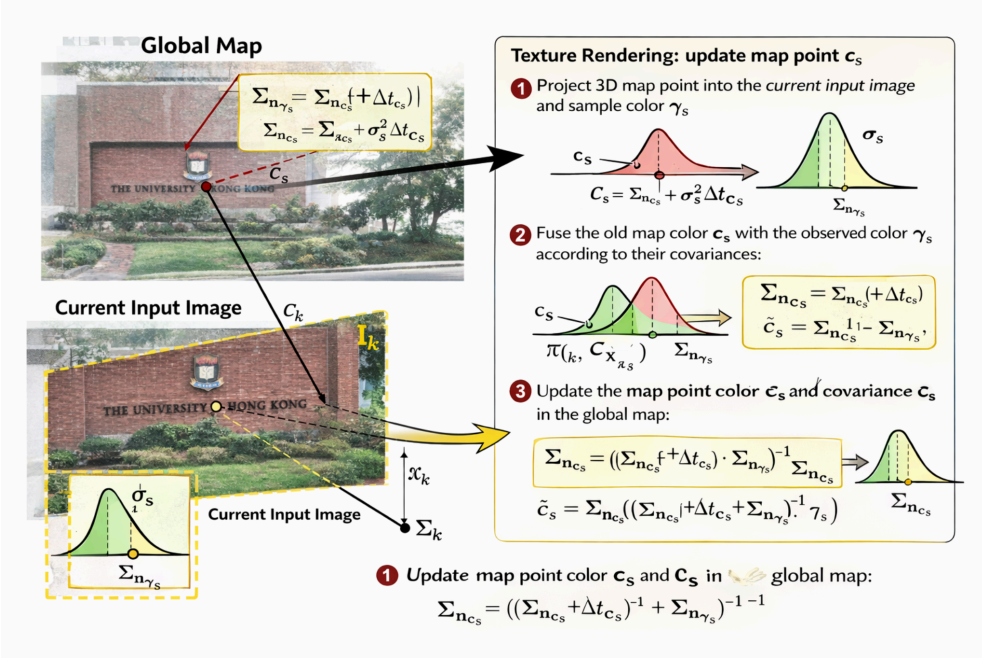
- 결국은 뭘 하고싶은것인가?
  1. LIO로 nominal state 구하기
  2. frame-to-frame photometric residual을 이용하여 ESIKF로 update
  3. frame-to-map phtometric residual을 이용하여 ESIKF로 update
  4. ESIKF는 Gauss-Newton 방식과 동일하게!



Method

Render the texture of global map

- 총 3단계의 pose estimation 단계를 거치면, 현재 pose에서 본 image도 정확해졌을 것
- 이 위치를 기반으로, map point의 RGB color를 이용하여, 현재 이미지에서 보이는 point들의 색을 render할 수 있음
- 즉, 기존 map의 색과, 현재 render한 색을 비교해서, 더 정확한 색으로 바꾸자!



Experiments

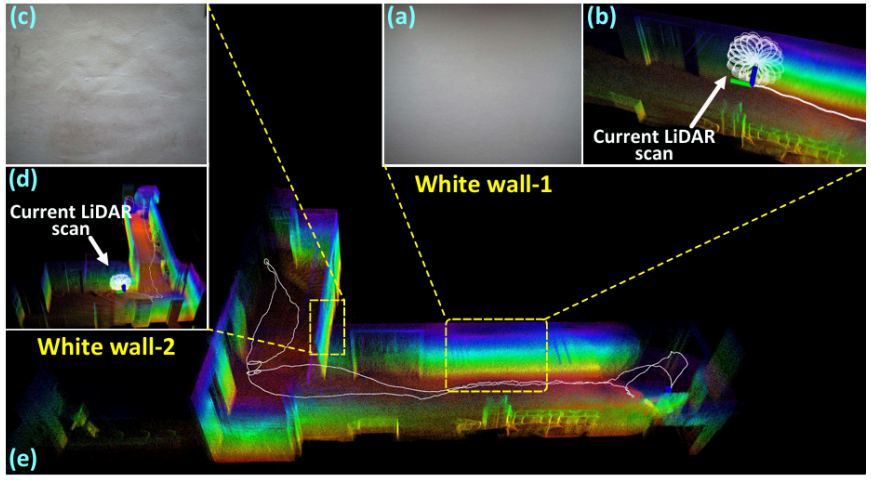
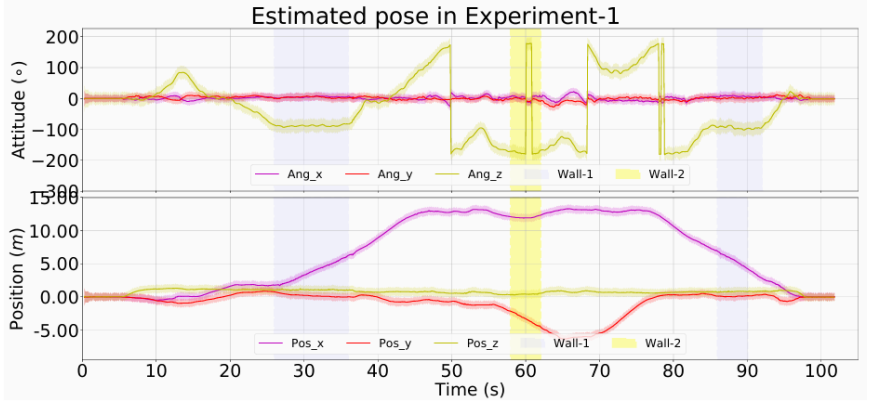


Fig. 7: We test our algorithm in simultaneously LiDAR degenerated and visual texture-less scenarios.



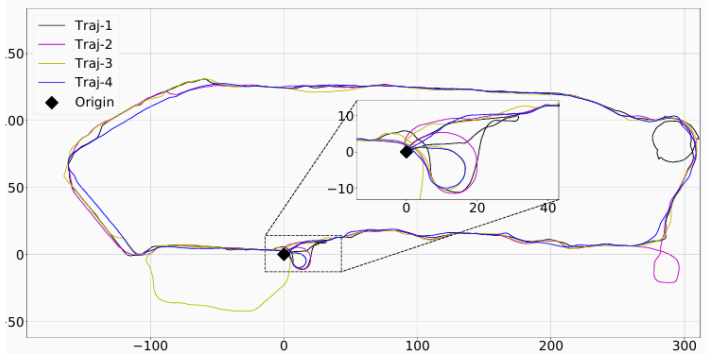


Fig. 10: The birdview of the 4 trajectories in Experiments-2, the total length of the four trajectories are 1317 m , 1524 m, 1372 m and 1191 m, respectively.

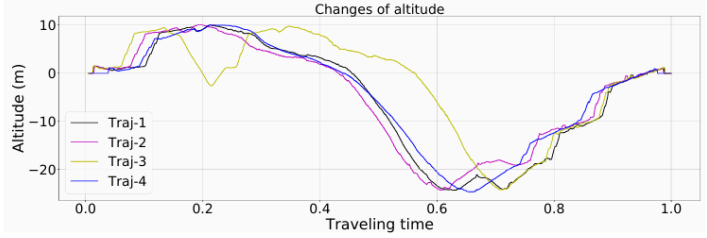


Fig. 11: The changes of altitude among the 4 trajectories in Experiment-2, where the time are normalized to 1.

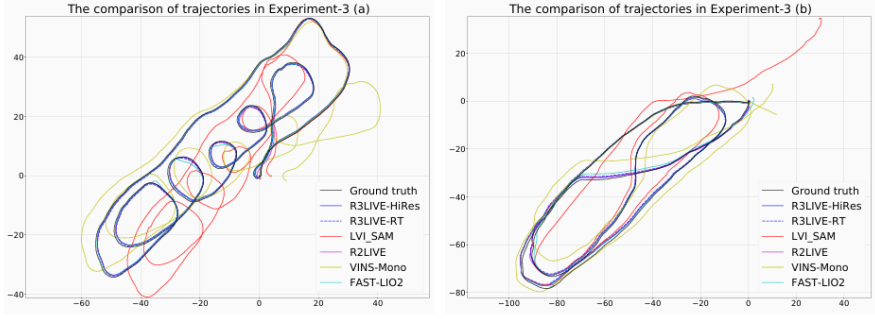


Fig. 12: Comparison of the estimated trajectories in Experiments-3.

Experiments

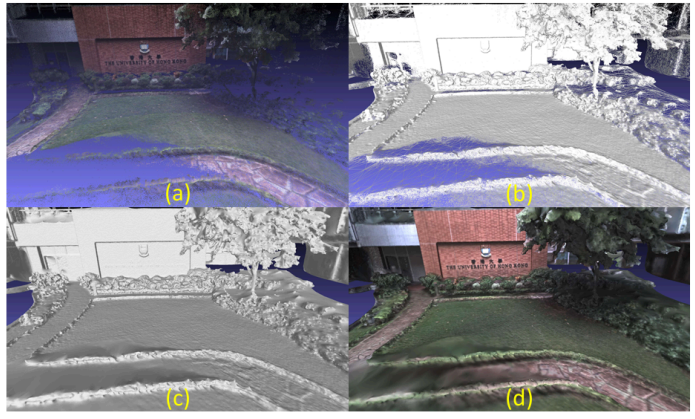


Fig. 13: Figure (a) show the RGB-colored 3D points reconstructed by R^{3}LIVE. Figure (b) and (c) show the wireframe and surface of our reconstructed mesh. Figure (d) show the mesh after texture rendering.

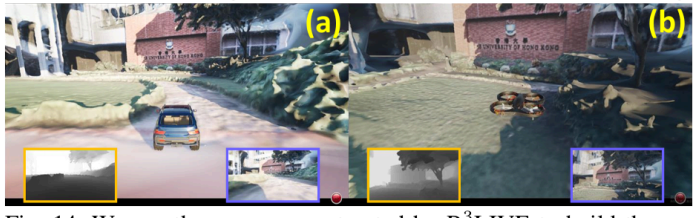


Fig. 14: We use the maps reconstructed by R^{3}LIVE to build the car (in (a)) and drone (in (b)) simulator with AirSim. The images in green and blue frameboxes are of the depth, RGB image query from the airsim's API, respectively.

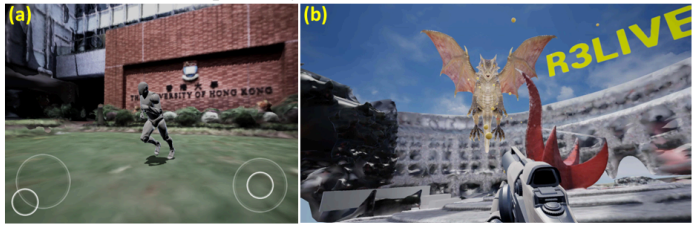


TABLE III: The relative pose error in Experiment-3. In configuration "R3LIVE-HiRES", we set the input image resolution as 1280×1024 and the minimum distance between two points in maps as 0.01 meters. In configuration "R3LIVE-RT", we set the input image resolution as 320×256 and the minimum distance between two points in maps as 0.10 meters

Relative pose error in Experiments-3 (a)						
Sub-sequence RRE/RTE	50 m deg / %	100 m deg / %	150 m deg / %	200 m deg / %	250 m deg / %	300 m deg / %
R3LIVE-HiRes	**0.99** / 2.25	**0.53** / **1.02**	**0.46** / **0.60**	**0.29** / **0.31**	**0.24** / 0.31	**0.21** / **0.17**
R3LIVE-RT	1.48 / **2.21**	0.54 / 1.06	0.49 / 0.60	0.31 / 0.41	0.25/ **0.31**	0.22 / 0.23
LVI_SAM	2.11 / 13.73	1.04 / 6.69	0.83 / 5.07	0.60 / 3.86	0.47 / 2.98	0.43 / 2.40
R2LIVE	1.21 / 2.47	0.61 / 1.02	0.59 / 0.60	0.34 / 0.34	0.37 / 0.33	0.34 / 0.21
FAST-LIO2	1.36 / 2.35	0.66 / 0.82	0.47 / 0.60	0.37 / 0.36	0.37 / 0.31	0.21 / 0.20
VINS-Mono	3.03 / 10.41	1.70 / 7.63	1.15 / 6.36	0.89 / 4.34	0.73 / 3.08	0.59 / 2.31
Relative pose error in Experiments-3 (b)						
Sub-sequence RRE/RTE	50 m deg / %	100 m deg / %	150 m deg / %	200 m deg / %	250 m deg / %	300 m deg / %
R3LIVE-HiRes	1.06 / **1.98**	0.58 / **1.34**	**0.43** / **0.83**	**0.32** / **0.59**	0.27 / **0.27**	0.25 / 0.16
R3LIVE-RT	**0.98** / 2.08	**0.55** / 1.61	0.47 / 0.99	0.39 / 0.65	0.33 / 0.33	0.25 / **0.14**
LVI_SAM	2.04 / 3.33	1.05 / 2.37	0.81 / 1.53	0.57 / 1.38	0.49 / 1.13	0.44 / 1.01
R2LIVE	1.13 / 2.06	0.65 / 1.45	0.49 / 0.88	0.31 / 0.57	0.26 / 0.30	0.29 / 0.16
FAST-LIO2	1.31 / 2.22	0.69 / 1.34	0.49 / 0.89	0.31 / 0.59	**0.26** / 0.31	**0.25** / 0.29
VINS-Mono	2.44 / 9.35	1.31 / 7.28	0.99 / 4.63	0.66 / 3.68	0.56 / 3.24	0.48 / 2.73

TABLE IV: The average time consumption on two different platforms with different configurations.

VIO per-frame cost time										LIO
Image size	320×256			640×512			1280×1024			per-frame
Pt_res (m)	0.10	0.05	0.01	0.10	0.05	0.01	0.10	0.05	0.01	cost time
On PC (ms)	7.01	10.41	33.55	11.53	14.68	39.75	13.63	17.58	43.71	18.40
On OB (ms)	15.00	26.45	84.22	26.05	42.32	123.62	33.38	55.10	166.08	39.56

Conclusion

- LiDAR-Camera-IMU sensor fusion을 통한 RGB-colored point cloud map 생성 알고리즘
- NeRF가 나오고, Instant-NGP가 나올 때, SLAM에선 여전히 이런일이 있었다.
  - 3D Gaussian Splatting win..
- ESIKF는 결국 optimization과 다를 바 없다?
- 진정한 tightly-coupled 방식은 아니다.
  - Geometry와 texture가 서로 영향을 주지 않는다. 이게 맞나?
- R3LIVE도 Covariance를 계속해서 다룬다, 3DGS와 연결지을 수 있는 방법은?